LAPORAN PRAKTIKUM

STRUKTUR DATA

“Double Linked List”

disusun Oleh:

Sofian Arba’i

2511533029

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kehadirat Allah SWT karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum pekan 6 dengan judul "Double Linked List" ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum serta sebagai sarana untuk meningkatkan pemahaman mengenai penggunaan struktur data Double Linked List.

Dalam praktikum pekan 6 mempelajari dasar-dasar Double Linked List, yaitu salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung dua arah. Setiap node pada Double Linked List memiliki data serta dua pointer, yaitu pointer yang menunjuk ke node sebelumnya (prev) dan pointer yang menunjuk ke node berikutnya (next). Praktikum ini membahas proses pembuatan Double Linked List, operasi dasar seperti traversal, penambahan node, penghapusan node, dan pencarian data.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika masih terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 12 Mei 2026

Sofian Arba'i

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Pengertian Double Linked List 3
2.1.1 Kelebihan dan Kekurangan Double Linked List 3
2.2 Implementasi Double Linked List pada Java 3
2.2.1 Class NodeDLL_2511533029 4
2.2.2 Class InsertDLL_2511533029 5
2.2.3 Class PenelusuranDLL_2511533029 7
2.2.4 Class HapusDLL_2511533029 9
BAB III PENUTUP 13
3.1 Kesimpulan 13
3.2 Saran 13
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data agar dapat diproses secara efisien. Salah satu bentuk struktur data linear yang penting untuk dipelajari adalah Double Linked List. Double Linked List merupakan struktur data yang terdiri dari kumpulan node, di mana setiap node menyimpan data serta dua buah pointer, yaitu pointer yang menunjuk ke node berikutnya (next) dan pointer yang menunjuk ke node sebelumnya (prev). Berbeda dengan Single Linked List yang hanya dapat ditelusuri ke satu arah, Double Linked List memungkinkan penelusuran data ke dua arah sehingga proses penambahan, penghapusan, dan penelusuran data dapat dilakukan dengan lebih fleksibel.

Pada praktikum pekan 6, saya mempelajari implementasi Double Linked List menggunakan bahasa pemrograman Java. Praktikum ini bertujuan untuk memahami cara kerja Double Linked List serta berbagai operasi dasar yang dapat dilakukan, seperti traversal maju dan mundur, penambahan node, penghapusan node, dan pencarian data. Dengan mempelajari Double Linked List, saya diharapkan mampu memahami salah satu struktur data dasar yang banyak digunakan dalam pengembangan program.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 6 tentang Double Linked List adalah sebagai berikut:

1. Memahami konsep dasar dan struktur node pada Double Linked List.

2. Mengimplementasikan operasi dasar Double Linked List, seperti penambahan, penghapusan, dan penelusuran node menggunakan bahasa Java.
3. Melatih kemampuan dalam menganalisis dan mengelola data menggunakan struktur data yang dapat ditelusuri ke dua arah.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 6 tentang Double Linked List adalah sebagai berikut:

1. Menambah pemahaman tentang struktur data Double Linked List dan cara kerjanya.
2. Meningkatkan kemampuan pemrograman Java, khususnya dalam penggunaan pointer next dan prev.
3. Membantu menyelesaikan permasalahan pengolahan data secara lebih fleksibel dan efisien.

BAB II
PEMBAHASAN

2.1 Pengertian Double Linked List

Double Linked List adalah sekumpulan node data yang terurut linear atau sekuensial dengan dua buah pointer yaitu prev dan next. Double Linked List adalah linked list dengan node yang memiliki data dan dua buah reference link (biasanya disebut next dan prev) yang menunjuk ke node sebelum dan node sesudahnya. Dengan demikian, proses penyisipan dan penghapusan node dapat dilakukan dengan lebih mudah dan cepat, serta penelusuran data dapat dilakukan baik dari awal ke akhir maupun dari akhir ke awal.

2.1.1 Kelebihan dan Kekurangan Double Linked List

Meskipun struktur data double linked list memiliki banyak kegunaan, berikut beberapa kelebihan dan kekurangan yang perlu diperhatikan sebelum menggunakannya.

Kelebihan:

1. Bisa bergerak maju dan mundur
2. Penghapusan node lebih mudah
3. Navigasi lebih fleksibel

Kekurangan:

1. Membutuhkan memori lebih besar
2. Struktur lebih kompleks

2.2 Implementasi Double Linked List pada Java

Berikut implementasi penggunaan Double Linked List pada bahasa pemrograman java sesuai dengan intruksi dosen dan asisten praktikum.

2.2.1 Class NodeDLL_2511533029

Kode Program pertama yaitu sebuah kode program yang berfungsi sebagai dasar pembentukan struktur data Double Linked List. Kelas ini mendefinisikan sebuah node yang memiliki tiga komponen utama, yaitu data, pointer ke node berikutnya (next_3029), dan pointer ke node sebelumnya (prev_3029). Melalui konstruktor yang disediakan, setiap node dapat dibuat dengan nilai data tertentu dan secara otomatis diinisialisasi dalam keadaan belum terhubung dengan node lain. Berikut Kode Program Node Double Linked List

```java
package pekan6_2511533029;

public class NodeDLL_2511533029 {
    // mendefinisikan kelas node
    int data_3029; // data
    NodeDLL_2511533029 next_3029; // pointer ke next node
    NodeDLL_2511533029 prev_3029; // pointer ke previous node

    // Konstruktor
    public NodeDLL_2511533029 (int data) {
        this.data_3029 = data;
        this.next_3029 = null;
        this.prev_3029 = null;
    }

}
```

Kode Program 2.1

struktur data Double Linked List. Setiap node menyimpan sebuah nilai data dan dua buah referensi, yaitu next_3029 yang menunjuk ke node berikutnya serta prev_3029 yang menunjuk ke node sebelumnya. Dengan adanya dua referensi tersebut, data dapat ditelusuri ke dua arah, yaitu dari awal ke akhir maupun dari akhir ke awal.

Pada bagian atribut, data_3029 bertipe int digunakan untuk menyimpan nilai yang ada pada node. Variabel next_3029 bertipe NodeDLL_2511533029 berfungsi sebagai pointer ke node selanjutnya, sedangkan prev_3029 bertipe NodeDLL_2511533029 berfungsi sebagai pointer ke node sebelumnya.

Konstruktor NodeDLL_2511533029(int data) dipanggil ketika objek node baru dibuat. Parameter data akan disimpan ke dalam atribut data_3029, sedangkan next_3029 dan prev_3029 diinisialisasi dengan nilai null.

2.2.2 Class InsertDLL_2511533029

Kode program selanjutnya merupakan kode program yang menunjukkan implementasi operasi penyisipan (insertion) pada Double Linked List, yaitu penambahan node di awal, di akhir, dan pada posisi tertentu. Program membuktikan bahwa Double Linked List memungkinkan manipulasi data secara fleksibel karena setiap node memiliki pointer ke node sebelumnya dan berikutnya. Berikut Kode program Insert Double Linked List.

```
package pekan6_2511533029;

public class Insert_DLL_2511533029 {
    // menambahkan node di awal dll
    static NodeDLL_2511533029 insertBegin_3029 (NodeDLL_2511533029 head_3029, int data_3029) {
        // buat node baru
        NodeDLL_2511533029 new_node_3029 = new NodeDLL_2511533029 (data_3029);
        // jadikan pointer nextnya head
        new_node_3029.next_3029 = head_3029;
        // jadikan pointer prev head ke new node
        if (head_3029 != null) {
            head_3029.prev_3029 = new_node_3029;
        }
        return new_node_3029;
    }
    // fungsi menambahkan node di akhir
    public static NodeDLL_2511533029 insertEnd_3029 (NodeDLL_2511533029 head_3029, int newData_3029) {
        // buat node baru
        NodeDLL_2511533029 newNode_3029 = new NodeDLL_2511533029 (newData_3029);
        // jika dll null jadikan head
        if (head_3029 == null) {
            head_3029 = newNode_3029;
        } else {
            NodeDLL_2511533029 curr_3029 = head_3029;
            while (curr_3029.next_3029 != null) {
                curr_3029 = curr_3029.next_3029;
            }
            curr_3029.next_3029 = newNode_3029;
            newNode_3029.prev_3029 = curr_3029;
        }
        return head_3029;
    }
    //fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511533029 insertAtPosition (NodeDLL_2511533029 head_3029, int pos_3029, int new_data_3029) {
        // buat node baru
        NodeDLL_2511533029 new_node_3029 = new NodeDLL_2511533029 (new_data_3029);
        if (pos_3029 == 1) {
            new_node_3029.next_3029 = head_3029;
            if (head_3029 != null) {
                head_3029.prev_3029 = new_node_3029;
            }
```

Kode Program 2.2

```java
            head_3029 = new_node_3029;
            return head_3029;
        }
        NodeDLL_2511533029 curr_3029 = head_3029;
        for (int i=1 ; i < pos_3029 - 1 && curr_3029 != null; i++) {
            curr_3029 = curr_3029.next_3029;
            }
            if (curr_3029 == null) {
                System.out.println("Posisi tidak ada");
                return head_3029;
            }
            new_node_3029.prev_3029 = curr_3029;
            new_node_3029.next_3029 = curr_3029.next_3029;
            curr_3029.next_3029 = new_node_3029;
            if (new_node_3029.next_3029 != null) {
                new_node_3029.next_3029.prev_3029 = new_node_3029;
            }
        return head_3029;
    }
    public static void printList_3029 (NodeDLL_2511533029 head_3029) {
        NodeDLL_2511533029 curr_3029 = head_3029;
        while (curr_3029 != null) {
            System.out.print(curr_3029.data_3029 + " <-> ");
            curr_3029 = curr_3029.next_3029;
        }
        System.out.println();
    }
    public static void main (String [] args) {
        // membuat dll 2 <-> 3 <-> 5
        NodeDLL_2511533029 head_3029 = new NodeDLL_2511533029 (2);
        head_3029.next_3029 = new NodeDLL_2511533029 (3);
        head_3029.next_3029.prev_3029 = head_3029;
        head_3029.next_3029.next_3029 = new NodeDLL_2511533029 (5);
        head_3029.next_3029.next_3029.prev_3029 = head_3029.next_3029;
        // cetak dll awal
        System.out.print("DLL Awal: ");
        printList_3029(head_3029);
        // tambah 1 di awal
        head_3029 = insertBegin_3029 (head_3029,1);
        System.out.println("Simpul 1 ditambah di awal : ");
        printList_3029 (head_3029);
```

Kode Program 2.3

```java
        // tambah 6 di akhir
        System.out.println("simpul 6 di tambah di akhir: ");
        int data_3029 = 6;
        head_3029 = insertEnd_3029 (head_3029, data_3029);
        printList_3029 (head_3029);
        // menambahkan node 4 di posisi 4
        System.out.println("tambah node 4 di posisi 4: ");
        int data2_3029 = 4;
        int pos_3029 = 4;
        head_3029 = insertAtPosition (head_3029, pos_3029, data2_3029);
        printList_3029(head_3029);
    }

}
```

Kode Program 2.4

Program Insert_DLL_2511533029 merupakan implementasi operasi penambahan node pada struktur data Double Linked List menggunakan bahasa Java. Program ini memanfaatkan kelas NodeDLL_2511533029 sebagai representasi node yang memiliki tiga

komponen utama, yaitu data, pointer ke node berikutnya (next_3029), dan pointer ke node sebelumnya (prev_3029). Pada program ini terdapat tiga method utama, yaitu insertBegin_3029() untuk menambahkan node di awal linked list, insertEnd_3029() untuk menambahkan node di akhir linked list, dan insertAtPosition() untuk menyisipkan node pada posisi tertentu. Selain itu, method printList_3029() digunakan untuk menampilkan seluruh isi linked list secara berurutan.

Pada method main(), program terlebih dahulu membuat Double Linked List awal dengan data 2 <-> 3 <-> 5. Selanjutnya program menambahkan node bernilai 1 di awal list sehingga menjadi 1 <-> 2 <-> 3 <-> 5, kemudian menambahkan node bernilai 6 di akhir list sehingga menjadi 1 <-> 2 <-> 3 <-> 5 <-> 6. Setelah itu, program menyisipkan node bernilai 4 pada posisi ke-4 sehingga susunan akhir menjadi 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6. Setiap perubahan yang terjadi akan ditampilkan ke layar menggunakan method printList_3029().

2.2.3 Class PenelusuranDLL_2511533029

Kode program java berikutnya untuk Double Linked List yaitu kode program penelusuran. Sebuah kode program yang menunjukkan penggunaan operasi traversal pada Double Linked List, yaitu penelusuran maju (forward traversal) dan penelusuran mundur (backward traversal). Program membuktikan bahwa Double Linked List memiliki keunggulan dibanding Single Linked List karena dapat menelusuri data ke dua arah berkat adanya pointer next dan prev pada setiap node.

```java
package pekan6_2511533029;

public class PenelusuranDLL_2511533029 {
    //fungsi penelusuran maju
    static void forwardTraversal_3029 (NodeDLL_2511533029 head_3029) {
        // memulai penelusuran dari head
        NodeDLL_2511533029 curr_3029 = head_3029;
        // lanjutkan sampai akhir
        while  (curr_3029 != null) {
            // print data
            System.out.print(curr_3029.data_3029 +" <-> ");
            // pindah ke node berikutnya
            curr_3029 = curr_3029.next_3029;
            }
        // print spasi
        System.out.println();
        }
        // fungsi penelusuran mundur
        static void backwardTraversal_3029 (NodeDLL_2511533029 tail_3029) {
            // mulai dari akhir
            NodeDLL_2511533029 curr_3029 = tail_3029;
            // lanjut sampai head
            while (curr_3029 != null) {
                // cetak data
                System.out.print(curr_3029.data_3029 + " <-> ");
                // pindah ke node sebelumnya
                curr_3029 = curr_3029.prev_3029;
            }
```

Kode Program 2.5

```java
            // cetak spasi
            System.out.println();
        }
        public static void main (String [] args) {
            // cetak dll
            NodeDLL_2511533029 head_3029 = new NodeDLL_2511533029 (1);
            NodeDLL_2511533029 second_3029 = new NodeDLL_2511533029 (2);
            NodeDLL_2511533029 third_3029 = new NodeDLL_2511533029 (3);

            head_3029.next_3029 = second_3029;
            second_3029.prev_3029 = head_3029;
            second_3029.next_3029 = third_3029;
            third_3029.prev_3029 = second_3029;

            System.out.println(" penelusuran maju: ");
            forwardTraversal_3029 (head_3029);

            System.out.println("penelusuran mundur: ");
            backwardTraversal_3029 (third_3029);
        }
}
```

Kode Program 2.6

Program PenelusuranDLL_2511533029 merupakan implementasi proses penelusuran (traversal) pada struktur data Double Linked List menggunakan bahasa Java. Program ini menggunakan kelas NodeDLL_2511533029 sebagai representasi node yang memiliki data serta dua pointer, yaitu next_3029 untuk menunjuk ke node berikutnya dan prev_3029 untuk menunjuk ke node sebelumnya. Pada program ini terdapat dua method utama, yaitu forwardTraversal_3029()

untuk melakukan penelusuran maju dari node pertama (head) menuju node terakhir, dan backwardTraversal_3029() untuk melakukan penelusuran mundur dari node terakhir (tail) menuju node pertama.

Method forwardTraversal_3029() dimulai dengan menyimpan node head ke dalam variabel curr_3029, kemudian menggunakan perulangan while untuk menampilkan data setiap node selama curr_3029 tidak bernilai null. Setelah data dicetak, pointer akan dipindahkan ke node berikutnya melalui curr_3029 = curr_3029.next_3029. Sebaliknya, method backwardTraversal_3029() dimulai dari node tail dan menampilkan data setiap node sambil bergerak ke node sebelumnya melalui curr_3029 = curr_3029.prev_3029. Dengan cara ini, data dapat ditelusuri dari belakang ke depan.

Pada method main(), program membuat tiga node dengan nilai 1, 2, dan 3, lalu menghubungkannya sehingga membentuk Double Linked List 1 <-> 2 <-> 3. Setelah itu, program memanggil forwardTraversal_3029() untuk menampilkan data secara berurutan dari awal ke akhir, dan memanggil backwardTraversal_3029() untuk menampilkan data dari akhir ke awal.

2.2.4 Class HapusDLL_2511533029

Kode program java terakhir mengenai Double Linked List yaitu kode program hapus/delete dll. Sebuah kode program yang menunjukkan implementasi operasi penghapusan (deletion) pada Double Linked List, yaitu penghapusan node di awal, di akhir, dan pada posisi tertentu. Program membuktikan bahwa Double Linked List memungkinkan proses penghapusan dilakukan secara efisien karena setiap node memiliki pointer ke node sebelumnya dan node berikutnya. Berikut kode program Hapus Double Linked List.

```java
package pekan6_2511533029;

public class HapusDLL_2511533029 {
    // fungsi menghapus node awal
    public static NodeDLL_2511533029 delHead_3029(NodeDLL_2511533029 head_3029) {
        if (head_3029 == null) {
            return null;
        }
        NodeDLL_2511533029 temp_3029 = head_3029;
        head_3029 = head_3029.next_3029;
        if (head_3029 != null) {
            head_3029.prev_3029 = null;
        }
            return head_3029;
    }
    // fungsi menghapus di akhir
    public static NodeDLL_2511533029 delLast_3029(NodeDLL_2511533029 head_3029) {
        if (head_3029 == null) {
            return null;
        }
        if (head_3029.next_3029 == null) {
            return null;
        }
        NodeDLL_2511533029 curr_3029 = head_3029;
        while (curr_3029.next_3029 != null) {
            curr_3029 = curr_3029.next_3029;
        }
        // update pointer previous node
        if (curr_3029.prev_3029 != null) {
            curr_3029.prev_3029.next_3029 = null;
        }
        return head_3029;

    }
    // fungsi menghapus node di posisi tertentu
    public static NodeDLL_2511533029 delPos_3029(NodeDLL_2511533029 head_3029, int pos_3029) {
        // jika DLL kosong
        if (head_3029 == null) {
            return head_3029;
        }
        NodeDLL_2511533029 curr_3029 = head_3029;
```

Kode Program 2.7

```java
        // telusuri sampai node yang akan dihapus
        for (int i = 1; curr_3029 != null && i < pos_3029; ++i) {
            curr_3029 = curr_3029.next_3029;
        }
        // jika posisi tidak ditemukan
        if (curr_3029 == null) {
            return head_3029;
        }
        // update pointer
        if (curr_3029.prev_3029 != null) {
            curr_3029.prev_3029.next_3029 = curr_3029.next_3029;
        }
        if (curr_3029.next_3029 != null) {
            curr_3029.next_3029.prev_3029 = curr_3029.prev_3029;
        }
        // jika yang dihapus head
        if (head_3029 == curr_3029) {
            head_3029 = curr_3029.next_3029;
        }
        return head_3029;
    }
    // fungsi mencetak DLL
    public static void printlist_3029 (NodeDLL_2511533029 head_3029) {
        NodeDLL_2511533029 curr_3029 = head_3029;
        while (curr_3029 != null) {
            System.out.print(curr_3029.data_3029 + " ");
            curr_3029 = curr_3029.next_3029;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        //buat sebuah DLL
        NodeDLL_2511533029 head_3029 = new NodeDLL_2511533029(1);
        head_3029.next_3029 = new NodeDLL_2511533029(2);
        head_3029.next_3029.prev_3029 = head_3029;
        head_3029.next_3029.next_3029 = new NodeDLL_2511533029(3);
        head_3029.next_3029.next_3029.prev_3029 = head_3029.next_3029;
        head_3029.next_3029.next_3029.next_3029 = new NodeDLL_2511533029(4);
        head_3029.next_3029.next_3029.next_3029.prev_3029 = head_3029.next_3029.next_3029;
        head_3029.next_3029.next_3029.next_3029.next_3029 = new NodeDLL_2511533029(5);
        head_3029.next_3029.next_3029.next_3029.next_3029.prev_3029 = head_3029.next_3029.next_3029.next_3029;
```

Kode Program 2.8

```java
        System.out.print("DLL Awal : ");
        printList_3029(head_3029);

        System.out.print("Setelah head dihapus : ");
        head_3029 = delHead_3029(head_3029);
        printList_3029(head_3029);

        System.out.print("Setelah node terakhir dihapus : ");
        head_3029 = delLast_3029(head_3029);
        printList_3029(head_3029);

        System.out.print("menghapus node ke 2 : ");
        head_3029 = delPos_3029(head_3029, 2);
        printList_3029(head_3029);
    }
}
```

Kode Program 2.9

Program HapusDLL_2511533029 merupakan implementasi operasi penghapusan node pada struktur data Double Linked List menggunakan bahasa Java. Program ini menggunakan kelas NodeDLL_2511533029 sebagai representasi node yang memiliki tiga komponen utama, yaitu data, pointer ke node berikutnya (next_3029), dan pointer ke node sebelumnya (prev_3029). Pada program ini terdapat tiga method utama, yaitu delHead_3029() untuk menghapus node pertama (head), delLast_3029() untuk menghapus node terakhir, dan delPos_3029() untuk menghapus node pada posisi tertentu. Selain itu, method printList_3029() digunakan untuk menampilkan seluruh isi linked list setelah setiap proses penghapusan dilakukan.

Method delHead_3029() bekerja dengan memindahkan head ke node berikutnya, kemudian mengatur pointer prev pada head baru menjadi null. Method delLast_3029() menelusuri linked list hingga mencapai node terakhir, lalu memutus hubungan antara node terakhir dengan node sebelumnya. Sementara itu, method delPos_3029() mencari node pada posisi yang ditentukan, kemudian memperbarui pointer next dan prev dari node di sekitarnya agar node tersebut terhapus dari rangkaian linked list. Dengan cara ini, data dapat dihapus tanpa mengganggu hubungan antar node lainnya.

Pada method main(), program terlebih dahulu membuat Double Linked List dengan data 1 <-> 2 <-> 3 <-> 4 <-> 5. Setelah itu, program menampilkan linked list awal, menghapus node pertama sehingga menjadi 2 <-> 3 <-> 4 <-> 5, menghapus node terakhir sehingga menjadi 2 <-> 3 <-> 4, dan terakhir menghapus node pada posisi ke-2 sehingga hasil akhirnya menjadi 2 <-> 4. Setiap perubahan ditampilkan ke layar menggunakan method printList_3029().

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Struktur Data pekan 6, dapat disimpulkan bahwa Double Linked List merupakan struktur data linear yang terdiri dari sekumpulan node, di mana setiap node memiliki data serta dua pointer, yaitu next yang menunjuk ke node berikutnya dan prev yang menunjuk ke node sebelumnya. Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan ke dua arah, yaitu dari awal ke akhir maupun dari akhir ke awal. Implementasi dilakukan menggunakan bahasa pemrograman Java melalui beberapa kelas, yaitu NodeDLL_2511533029, Insert_DLL_2511533 029, PenelusuranDLL_2511533029, dan HapusDLL_2511533029.

Melalui praktikum ini, penulis memperoleh pemahaman bahwa Double Linked List memiliki keunggulan dalam hal fleksibilitas dan efisiensi dalam proses penyisipan, penghapusan, serta penelusuran data dibandingkan Single Linked List. Namun, struktur ini juga membutuhkan memori yang lebih besar karena setiap node menyimpan dua buah pointer.

3.2 Saran

Dalam pelaksanaan praktikum berikutnya, disarankan agar mahasiswa lebih memahami konsep dasar pointer dan hubungan antar node sebelum mengimplementasikan program Double Linked List. Selain itu, mahasiswa perlu lebih teliti dalam memperbarui pointer next dan prev, karena kesalahan kecil dapat menyebabkan struktur linked list menjadi tidak valid.

DAFTAR PUSTAKA

[1] Rizki Doank, “Double Linked List,”. [Daring]. Tersedia pada: Double Linked List | rizkidoank [Diakses: 12-Mei-2026].

[2] Geeks for Geeks, “Doubly Linked List Tutorial,”. [Daring]. Tersedia pada: Doubly Linked List Tutorial - GeeksforGeeks [Diakses: 12-Mei-2026].

[3] Diana Ape, “Linked List: Single, Double, dan Circular Linked List,”. [Daring]. Tersedia pada: Linked List: Single, Double, dan Circular Linked List [Diakses: 12-Mei-2026].

LAPORAN TUGAS PRAKTIKUM

Struktur Data

Pekan 6 (Double Linked List)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Tugas Double Linked List

Tema: Playlist Lagu Sederhana

Deskripsi Tugas

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

JAWABAN

1. Kode Program

Berdasarkan intruksi yang terdapat pada file, kode program tugas Double Linked List dibuat menjadi dua class. Class lagu dan class musik.

1.1 Class Musik_2511533029

Kode program music adalah sebuah kode program bagaimana pembentukan node pada Double Linked List untuk menyimpan data lagu dalam playlist musik. Setiap node menyimpan informasi judul dan penyanyi, serta memiliki pointer next dan prev yang memungkinkan hubungan antar lagu terbentuk secara dua arah. Berikutk ode program musik.

```java
package pekan6_2511533029;

public class Lagu_2511533029 {
    // atribut node
    private String judul_3029;
    private String penyanyi_3029;
    Lagu_2511533029 next_3029;
    Lagu_2511533029 prev_3029;

    // constructor
    public Lagu_2511533029(String judul_3029, String penyanyi_3029) {
        this.judul_3029 = judul_3029;
        this.penyanyi_3029 = penyanyi_3029;
        this.next_3029 = null;
        this.prev_3029 = null;
    }

    // getter
    public String getJudul_3029() {
        return judul_3029;
    }

    public String getPenyanyi_3029() {
        return penyanyi_3029;
    }

    // setter
    public void setJudul_3029(String judul_3029) {
        this.judul_3029 = judul_3029;
    }

    public void setPenyanyi_3029(String penyanyi_3029) {
        this.penyanyi_3029 = penyanyi_3029;
    }

}
```

Kode Program 1.1

Program Lagu_2511533029 merupakan kelas yang digunakan untuk merepresentasikan satu node pada struktur data Double Linked List dalam program playlist musik. Setiap objek dari kelas ini menyimpan informasi tentang sebuah lagu, yaitu judul lagu dan nama penyanyi, serta dua buah pointer yang menghubungkan node dengan node lainnya. Pointer next_3029 digunakan untuk menunjuk ke node berikutnya, sedangkan pointer prev_3029 digunakan untuk menunjuk ke node sebelumnya.

Konstruktor Lagu_2511533029 (String judul_3029, String penyanyi_3029) digunakan untuk membuat objek lagu baru dengan mengisi nilai judul dan penyanyi sesuai parameter yang diberikan. Saat objek dibuat, pointer next_3029 dan prev_3029 secara otomatis diinisialisasi dengan nilai null, yang menandakan bahwa node tersebut belum terhubung dengan node lain. Selain itu, kelas ini menyediakan method getter getJudul_3029() dan getPenyanyi_3029() untuk mengambil nilai atribut, serta method setter setJudul_3029() dan setPenyanyi_3029() untuk mengubah nilai atribut jika diperlukan.

1.2 Class Lagu_2511533029

Pada kode program class lagu adalah kode program yang digunakan untuk menunjukkan bagaimana node pada double linked list dalam bentuk aplikasi pengelolaan playlist musik. Program memanfaatkan node yang saling terhubung melalui pointer next dan prev untuk mendukung operasi penambahan, penghapusan, penelusuran dua arah, dan pencarian data. Berikut kode program class lagu.

```
package pekan6_2511533029;
import java.util.Scanner;

public class Musik_2511533029 {
 // pointer head dan tail
    Lagu_2511533029 head_3029 = null;
    Lagu_2511533029 tail_3029 = null;

    // 1. menambah lagu di akhir playlist
```

```
10.     public void tambahLagu_3029(String judul_3029, String
    penyanyi_3029) {
11.         Lagu_2511533029 laguBaru_3029 =
12.                 new Lagu_2511533029(judul_3029,
    penyanyi_3029);
13.
14.         // jika playlist kosong
15.         if (head_3029 == null) {
16.             head_3029 = laguBaru_3029;
17.             tail_3029 = laguBaru_3029;
18.         } else {
19.             tail_3029.next_3029 = laguBaru_3029;
20.             laguBaru_3029.prev_3029 = tail_3029;
21.             tail_3029 = laguBaru_3029;
22.         }
23.
24.         System.out.println("Lagu berhasil ditambahkan!");
25.     }
26.
27.     // 2. menghapus lagu pertama
28.     public void hapusLaguAwal_3029() {
29.         if (head_3029 == null) {
30.             System.out.println("Playlist kosong!");
31.             return;
32.         }
33.
34.         System.out.println("Lagu \"" +
    head_3029.getJudul_3029()
35.                 + "\" berhasil dihapus.");
36.
37.         // jika hanya ada satu node
38.         if (head_3029 == tail_3029) {
39.             head_3029 = null;
40.             tail_3029 = null;
41.         } else {
42.             head_3029 = head_3029.next_3029;
43.             head_3029.prev_3029 = null;
44.         }
45.     }
46.
47.     // 3. menampilkan playlist dari awal ke akhir
48.     public void tampilMaju_3029() {
49.         if (head_3029 == null) {
50.             System.out.println("Playlist kosong!");
51.             return;
52.         }
53.
54.         Lagu_2511533029 curr_3029 = head_3029;
55.         int nomor_3029 = 1;
56.
57.         while (curr_3029 != null) {
58.             System.out.println(nomor_3029 + ". "
59.                     + curr_3029.getJudul_3029()
60.                     + " - "
```

```
                        + curr_3029.getPenyanyi_3029());
                curr_3029 = curr_3029.next_3029;
                nomor_3029++;
            }
        }

        // 4. menampilkan playlist dari akhir ke awal
        public void tampilMundur_3029() {
            if (tail_3029 == null) {
                System.out.println("Playlist kosong!");
                return;
            }

            Lagu_2511533029 curr_3029 = tail_3029;
            int nomor_3029 = 1;

            while (curr_3029 != null) {
                System.out.println(nomor_3029 + ". "
                        + curr_3029.getJudul_3029()
                        + " - "
                        + curr_3029.getPenyanyi_3029());
                curr_3029 = curr_3029.prev_3029;
                nomor_3029++;
            }
        }

        // 5. mencari lagu berdasarkan judul (tidak case-sensitive)
        public void cariLagu_3029(String judul_3029) {
            if (head_3029 == null) {
                System.out.println("Playlist kosong!");
                return;
            }

            Lagu_2511533029 curr_3029 = head_3029;
            boolean ditemukan_3029 = false;

            while (curr_3029 != null) {
                if (curr_3029.getJudul_3029()
                        .equalsIgnoreCase(judul_3029)) {
                    System.out.println("Lagu ditemukan:");
                    System.out.println("Judul    : "
                            + curr_3029.getJudul_3029());
                    System.out.println("Penyanyi : "
                            + curr_3029.getPenyanyi_3029());
                    ditemukan_3029 = true;
                    break;
                }
                curr_3029 = curr_3029.next_3029;
            }

            if (!ditemukan_3029) {
                System.out.println("Lagu tidak ditemukan.");
            }
```

```
    }

    // menu utama
    public static void main(String[] args) {
        Scanner input_3029 = new Scanner(System.in);
        Musik_2511533029 playlist_3029 =
                new Musik_2511533029();

        int pilihan_3029;

        do {
            System.out.println("\n=== Playlist Musik NIM: 2511533029 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan_3029 = input_3029.nextInt();
            input_3029.nextLine();

            switch (pilihan_3029) {
                case 1:
                    System.out.print("Judul: ");
                    String judul_3029 = input_3029.nextLine();

                    System.out.print("Penyanyi: ");
                    String penyanyi_3029 =
                            input_3029.nextLine();

                    playlist_3029.tambahLagu_3029(
                            judul_3029,
                            penyanyi_3029
                    );
                    break;

                case 2:
                    playlist_3029.hapusLaguAwal_3029();
                    break;

                case 3:
                    System.out.println("\nPlaylist (Maju):");
                    playlist_3029.tampilMaju_3029();
                    break;

                case 4:
                    System.out.println("\nPlaylist (Mundur):");
                    playlist_3029.tampilMundur_3029();
```

```
                    break;

                case 5:
                    System.out.print("Masukkan judul lagu: ");
                    String cari_3029 =
                            input_3029.nextLine();
                    playlist_3029.cariLagu_3029(cari_3029);
                    break;

                case 6:
                    System.out.println("Program selesai.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan_3029 != 6);

        input_3029.close();
    }

}
```

2. Output

Berikut beberapa output berdasarkan 2 class kode program, class lagu dan class musik.

```
=== Playlist Musik NIM: 2511533029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Tentang Rasa
Penyanyi: Astrid
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Janji Setia
Penyanyi: Tiara Andini
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

Playlist (Maju):
1. Tentang Rasa - Astrid
2. Janji Setia - Tiara Andini
```

```
=== Playlist Musik NIM: 2511533029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4

Playlist (Mundur):
1. Janji Setia - Tiara Andini
2. Tentang Rasa - Astrid

=== Playlist Musik NIM: 2511533029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: Janji Setia
Lagu ditemukan:
Judul    : Janji Setia
Penyanyi : Tiara Andini

=== Playlist Musik NIM: 2511533029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu "Tentang Rasa" berhasil dihapus.
```

```
=== Playlist Musik NIM: 2511533029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai.
```